

Name : Rushikesh Bhokare

batch :-B4

Class: sy-b(67).

Aim - 2. Data Ingestion and Cleaning: Acquire data and prepare it for analysis.

THEORY

1. What is data ingestion and why is it important? Which Python library is used?

Data ingestion is the process of collecting and importing data from different sources such as CSV files, databases, or APIs into a system for analysis. It is an important step in data science because it provides the raw data required for further processing and analysis. Without data ingestion, no analysis or machine learning can be performed. In Python, the most commonly used library for data ingestion and cleaning is Pandas, which provides easy functions to read, manipulate, and process data.

1. Why is data cleaning necessary before analysis or machine learning?

Data cleaning is necessary because real-world data often contains errors, missing values, duplicates, and inconsistent formats. If this unclean data is used directly, it can lead to incorrect results and poor model performance. Cleaning the data ensures accuracy, consistency, and reliability, which improves the quality of analysis and machine learning models. Therefore, data cleaning is a crucial step before any data processing or analysis.

1. How do you load a CSV file into a Pandas DataFrame?

A CSV file can be loaded into a Pandas DataFrame using the `read_csv()` function. First, the Pandas library is imported, and then the file is read by specifying its path. The data is stored in a DataFrame, which allows easy manipulation and analysis. This method is simple and widely used for handling structured data in Python.

1. What is the difference between `head()` and `tail()` functions in Pandas?

The `head()` and `tail()` functions are used to view a portion of the dataset. The `head()` function displays the first few rows of the DataFrame, usually the first five rows by default, which helps in quickly understanding the structure of the data. On the other hand, the `tail()` function displays the last few rows of the dataset. Both functions are useful for inspecting data without displaying the entire dataset.

```
In [ ]: from google.colab import files
        uploaded = files.upload()
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving employee_performance_data (1).csv to employee_performance_data (1).csv

```
In [ ]: # Step 2: Import pandas
import pandas as pd
```

```
In [ ]: # Step 3: Get file name
file_name = list(uploaded.keys())[0]
```

```
In [ ]: # Step 4: Read CSV
df = pd.read_csv(file_name)
```

```
In [ ]: # Step 5: Show data
print("First 5 rows:")
print(df.head())
```

First 5 rows:

	Employee_ID	Name	Age	Gender	Department	Join_Date	Salary \
0	101	Alice	50	Female	Marketing	2009-01-18	62606.0
1	102	Bob	36	Female	Marketing	2012-01-01	41534.0
2	103	Charlie	29	Female	IT	2007-02-15	NaN
3	104	David	42	Male	Marketing	2015-12-29	31016.0
4	105	Eve	40	Female	IT	2005-02-04	119789.0

	Experience	Performance_Rating	Bonus	Work_Hours_Per_Week
0	13.0	2.0	9488.0	44.0
1	1.0	1.0	5206.0	40.0
2	9.0	4.0	10134.0	33.0
3	7.0	1.0	10977.0	42.0
4	9.0	4.0	12721.0	36.0

```
In [ ]: # Step 6: Missing values
print("\nMissing values:")
print(df.isnull().sum())
```

Missing values:

Employee_ID	0
Name	0
Age	0
Gender	0
Department	0
Join_Date	0
Salary	1
Experience	1
Performance_Rating	1
Bonus	1
Work_Hours_Per_Week	1

dtype: int64

```
In [ ]: # Step 7: Fill missing values
df = df.fillna(df.mean(numeric_only=True))
```

```
In [ ]: # Step 8: Lowercase columns
df.columns = df.columns.str.lower()
```

```
In [ ]: # display dataset shape
print(df.shape)
```

(20, 11)

```
In [ ]:
```

```
# Step 9: Remove duplicates
df = df.drop_duplicates()
```

```
In [ ]: # Handle missing values (mean/median)
df = df.fillna(df.mean(numeric_only=True)) # mean
df = df.fillna(df.median(numeric_only=True)) # median
```

```
In [ ]: # Change data types
# Replace 'salary' with the actual column name you intend to convert.
if 'salary' in df.columns:
    df['salary'] = df['salary'].astype('int')
    print("Datatype changed successfully")
    print(df.dtypes)
else:
    print("Column 'salary' not found")
```

```
Datatype changed successfully
employee_id      int64
name             object
age             int64
gender           object
department       object
join_date        object
salary           int64
experience       float64
performance_rating float64
bonus           float64
work_hours_per_week float64
dtype: object
```

```
In [ ]: #Filter rows
filtered = df[df['age'] > 50]
print(filtered)
```

	employee_id	name	age	gender	department	join_date	salary	\
9	110	Jack	57	Male	Marketing	2013-06-01	104065	
15	116	Paul	51	Male	IT	2009-02-09	116779	
16	117	Quinn	59	Female	HR	2018-02-20	69099	
19	120	Tina	54	Female	Marketing	2010-06-05	81214	

	experience	performance_rating	bonus	work_hours_per_week
9	8.0	4.0	7790.0	35.0
15	3.0	3.0	6757.0	30.0
16	1.0	1.0	19820.0	30.0
19	10.0	3.0	11892.0	38.0

```
In [ ]: #Save cleaned dataset
df.to_csv("cleaned_data.csv", index=False)
```